

Práctica Hospital

```
31 def __init__(self, settings):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, "requests.log"),
39                         "a")
40         self.file.seek(0)
41         self.fingerprints.update(self.request_fingerprint(request))
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool("DEBUG_LOG_REQUESTS")
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```



Unai Celaya (29 de diciembre de 2024)

Índice

- 1. Arquitectura utilizada.....página 3-4**
- 2. Diagrama de la arquitectura.....página 5**
- 3. Pseudocódigo.....página 6**
- 4. Eficiencia del código.....página 7**
- 4. Bibliografía.....página 8**

Arquitectura utilizada:

El ejercicio plantea un sistema de gestión hospitalaria, donde los pacientes y doctores interactúan a través de una aplicación para registrar consultas y recetar medicamentos. Como Arquitectura utilizada he planteado una basada en Cliente-Servidor y Peer-to-Peer (P2P).

En este tipo de arquitectura, los nodos en la red actúan tanto como clientes como servidores. No hay un servidor centralizado, y los nodos se comunican y comparten recursos directamente entre sí. Es comúnmente utilizado en aplicaciones de intercambio de archivos y sistemas distribuidos. En el contexto de la gestión hospitalaria, esto significa que los ordenadores de todos los doctores están conectados entre sí, permitiendo la actualización y sincronización de los datos de los pacientes en tiempo real.

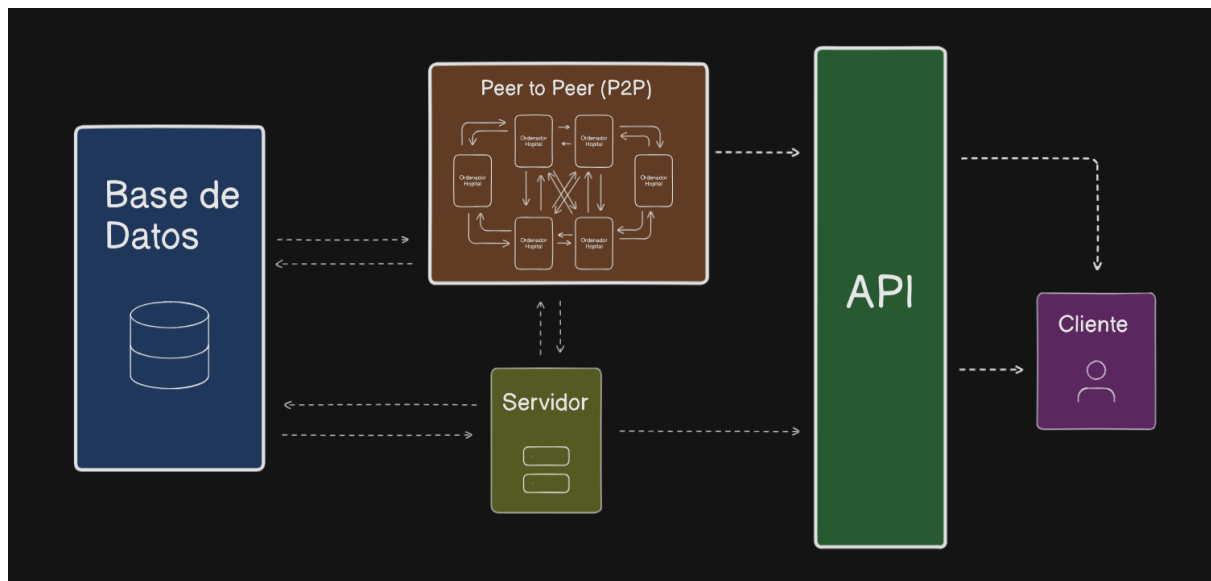
En esta arquitectura, al no depender de un único servidor central, la red es más resiliente a fallos. Si uno o varios nodos fallan, los demás pueden continuar operando y compartiendo datos. La carga de procesamiento y almacenamiento se distribuye entre todos los nodos de la red. Esto puede resultar en un uso más eficiente de los recursos disponibles. También tiene mucho impacto en la posibilidad de escalar ya que a medida que se añaden más nodos (más doctores o ordenadores de trabajo), la capacidad del sistema aumenta, permitiendo manejar un mayor volumen de datos y usuarios sin comprometer el rendimiento general del sistema. La arquitectura P2P facilita la colaboración directa entre doctores, permitiendo el acceso y actualización instantánea de los historiales clínicos compartidos. Esto es especialmente útil en situaciones donde varios especialistas deben trabajar juntos en el tratamiento de un paciente.

Por otro lado, la arquitectura Cliente-Servidor es un modelo más tradicional en el que un servidor centralizado gestiona las solicitudes de múltiples clientes. Se basa en la separación de responsabilidades entre el cliente y el servidor. El cliente solicita servicios o recursos al servidor, que los proporciona y responde a las peticiones. En este sistema, los clientes (doctores y personal del hospital) se conectan al servidor para acceder, actualizar y gestionar los datos de los pacientes. Esta arquitectura permite la escalabilidad y distribución de los componentes del sistema

Esta arquitectura, tiene un servidor central que facilita el control total sobre la base de datos de pacientes, consultas y recetas médicas. Esto garantiza que toda la información sea accesible. En esta arquitectura es más sencillo implementar medidas de seguridad, como la autenticación de usuarios para proteger los datos contra accesos no autorizados. El servidor puede gestionar de manera efectiva el acceso a los datos, asegurando que solo personal autorizado pueda acceder a información sensible

La mezcla de estas dos arquitecturas podría permitir a los hospitales manejar grandes volúmenes de datos de manera eficiente, ya que todos los ordenadores estarán conectados y guardarán la información de manera segura en un servidor central usando así lo mejor de cada arquitectura, pudiendo evitar cuelgues del servidor ya que todos los ordenadores funcionan de manera independiente y conectada.

Diagrama de la arquitectura:



(Diagrama arquitectura)

Aquí podemos ver el diagrama de la arquitectura planteada para hacer el ejercicio, esta sería la mezcla entre el peer to peer (P2P) y la arquitectura de cliente servidor, donde tenemos una base de datos centralizada para guardar la información de manera segura y el proceso donde todos los ordenadores están conectados para trabajar con los datos de los pacientes.

Pseudocódigo

Crear receta:

Función recetar_medimento(paciente, medicamento):

Si el paciente no está en pacientes_asignados:

Lanzar excepción "El paciente no está asignado a este doctor."

Crear una nueva consulta con los siguientes datos:

fecha = fecha_actual()

síntomas = "N/A"

diagnóstico = "Receta directa"

medicamento = medicamento

Agregar la consulta al historial del paciente

Retornar el mensaje "Medicamento {medicamento} recetado a {paciente.nombre}."

Fin de la función

Eficiencia del código

En este caso, la función asignar paciente _a_ doctor utiliza una lista (self.pacientes) para almacenar los pacientes. Las listas en Python tienen una eficiencia de **$O(n)$** (Big O) donde Big O Indica el peor escenario posible en términos de tiempo de ejecución. Describe la cota superior asintótica, es decir, la tasa de crecimiento máxima del tiempo de ejecución en función del tamaño de entrada. Para las operaciones de búsqueda, lo que significa que, para encontrar un paciente, en el peor de los casos, se tendrá que recorrer toda la lista de pacientes

Bibliografía

Eraser. (2025, enero 8). *Eraser AI*.

<https://www.eraser.io/ai>

EUNEIZ. (n.d.). *Canvas*. Recuperado el 28 de diciembre de 2024, de

<https://canvas.euneiz.com/>

OpenAI. (n.d.). *Chat GPT*. Recuperado el 28 de diciembre de 2024, de

<https://chatgpt.com/>